

HIGH PRESSURE DETECTION AND ALARM SYSTEM

System Modeling and Design Report Using TTool

Frist Term (Final Project)

Prepared by: Mohamed Aldreamly

Diploma: Mastering Embedded System Online Diploma

Platform: Learn In Depth

Platform Link: <https://www.learn-in-depth-store.com/account/my-account>

TTool / AVATAR Modeling

Executive Summary

This report presents the TTool model of a high-pressure detection and alarm system. The system periodically reads a pressure value from a sensor, compares the measured value against a threshold of 20 bar, and activates an alarm path when the threshold condition is met. The alarm remains active for 60 seconds before being stopped and reset. The pressure sensor driver is modeled with a 100-second polling period. An optional local data-storage path is also represented in the analysis models to record pressure values and system behavior.

The model is described through requirement, use-case, activity, sequence, AVATAR block, and state-machine diagrams. Together, these diagrams show the system requirements, component responsibilities, signal exchange, timing behavior, and end-to-end execution flow.

1. Introduction

High-pressure conditions can represent a hazardous operating state in many embedded applications. The purpose of this project is to model a simple embedded monitoring system that can detect an abnormal pressure level and trigger a timed alarm response. TTool is used to capture both the analysis and design views of the system, while the AVATAR design model is used to describe the internal blocks and their state-based behavior.

2. System Objectives

- Read the pressure value from the pressure-sensor interface.
- Compare the measured pressure with a configured threshold of 20 bar.
- Trigger the alarm path when the high-pressure condition is detected.
- Keep the alarm active for 60 seconds, then stop and reset it.
- Repeat sensor acquisition using a 100-second polling period.
- Represent optional local storage of measured pressure values for future implementation or logging.

3. Functional Requirements

ID	Requirement	Description
R0	High-pressure protection	The system provides protection logic against high-pressure conditions.
R1	High-pressure detection	The system checks the pressure condition in the monitored environment.
R2	Crew / user information	The system informs the user when a high-pressure condition is detected.
R3	Data storage	The model includes a path for storing system information.
R4	Pressure threshold	The pressure threshold is modeled as 20 bar.
R5	Sensor acquisition	The system reads pressure data from the pressure sensor.
R6	Alarm behavior	The alarm is activated and maintained for a 60-second period.
R7	Flash-memory logging	Recorded values can be stored in local flash memory as an optional feature.

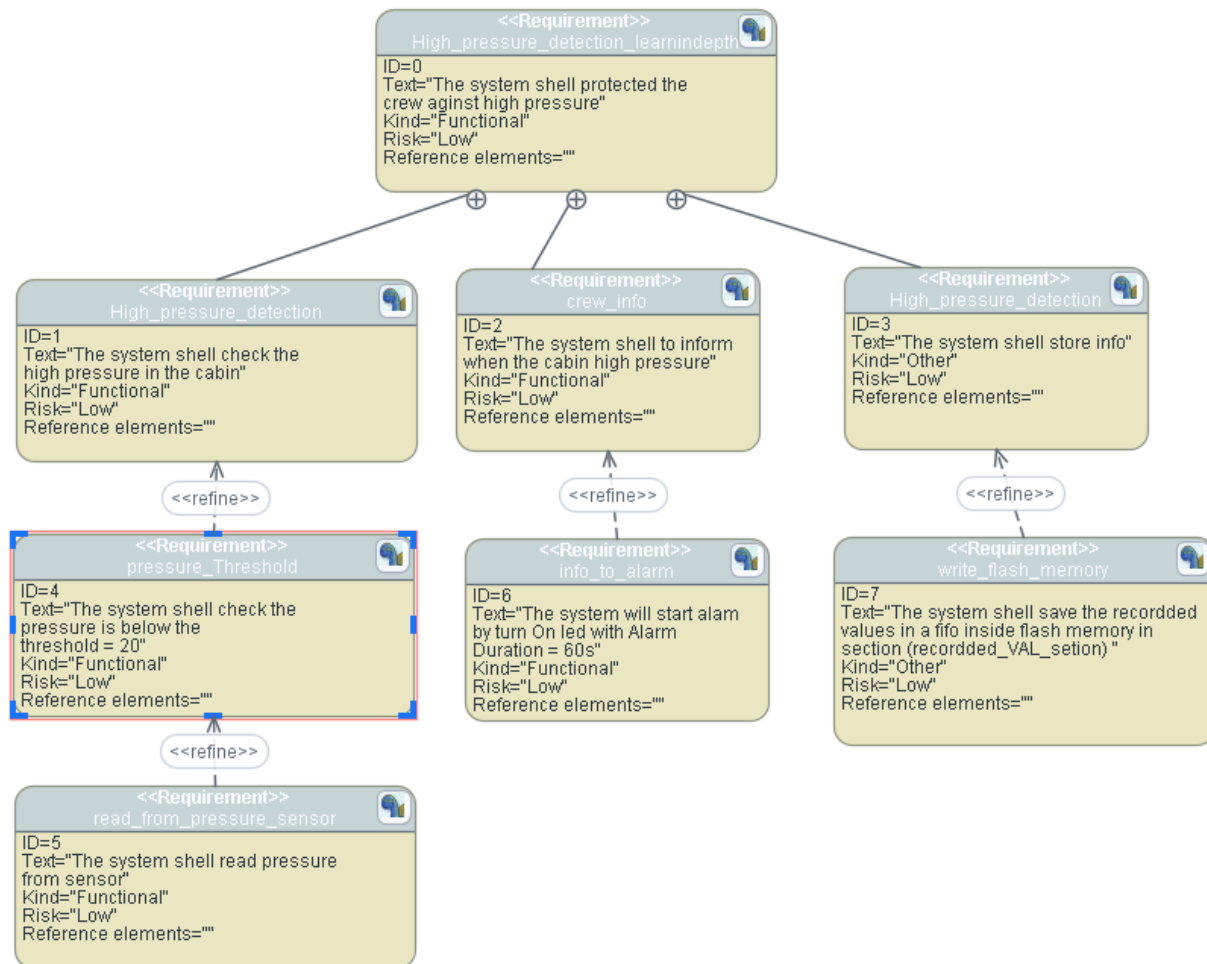


Figure 1. Requirements diagram for the high-pressure detection system.

4. System Analysis

4.1 Use-Case View

The use-case model identifies the external interfaces and the main system functions. The pressure sensor supplies the measured value, the main algorithm evaluates the pressure, the alarm-monitor function manages alarm timing, the alarm actuator represents the physical warning output, and the flash-memory actor represents the optional storage destination.

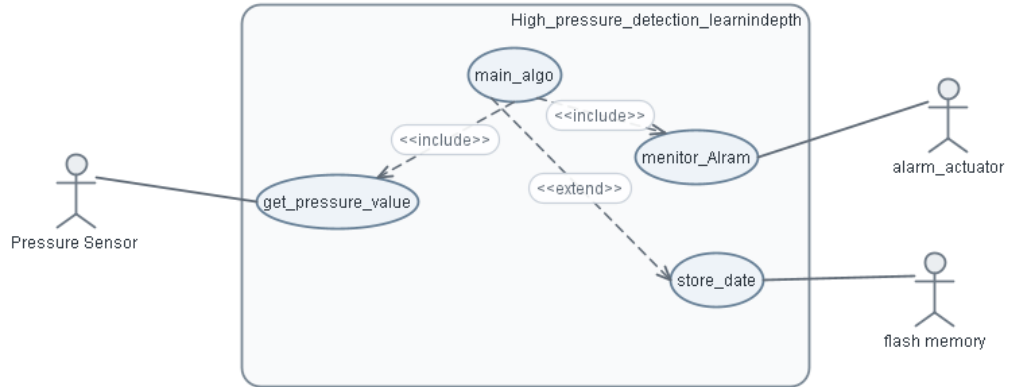


Figure 2. Use-case diagram showing the system actors and main functions.

4.2 Activity Flow

The activity model summarizes the operational flow. A pressure value is acquired, optional storage may be performed, and the pressure condition is evaluated. When the alarm condition is reached, the system starts the alarm, waits for the configured alarm period, turns the alarm off, and then returns to the monitoring cycle.

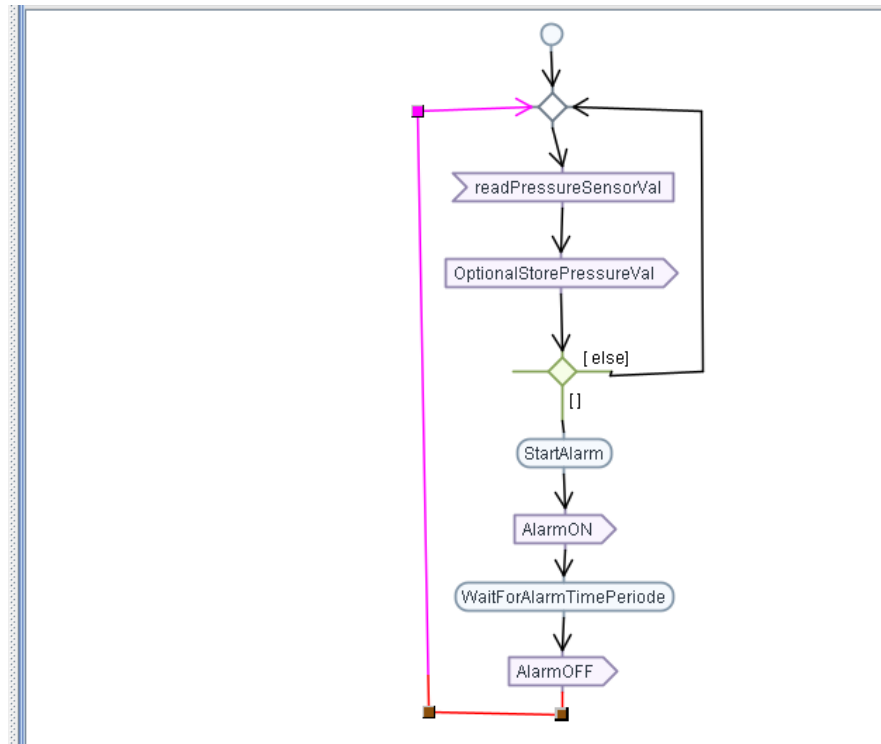


Figure 3. Activity diagram of the pressure-monitoring and alarm process.

4.3 Interaction Sequence

The sequence diagram illustrates a representative scenario. A normal pressure value such as 12 bar is delivered to the main algorithm and may be stored without activating the alarm. A later value such as 22 bar results in an alarm-start request. The alarm monitor starts the 60-second timer, later sends the stop-alarm command, and resets the timer. The same interaction also shows the optional storage of pressure values.

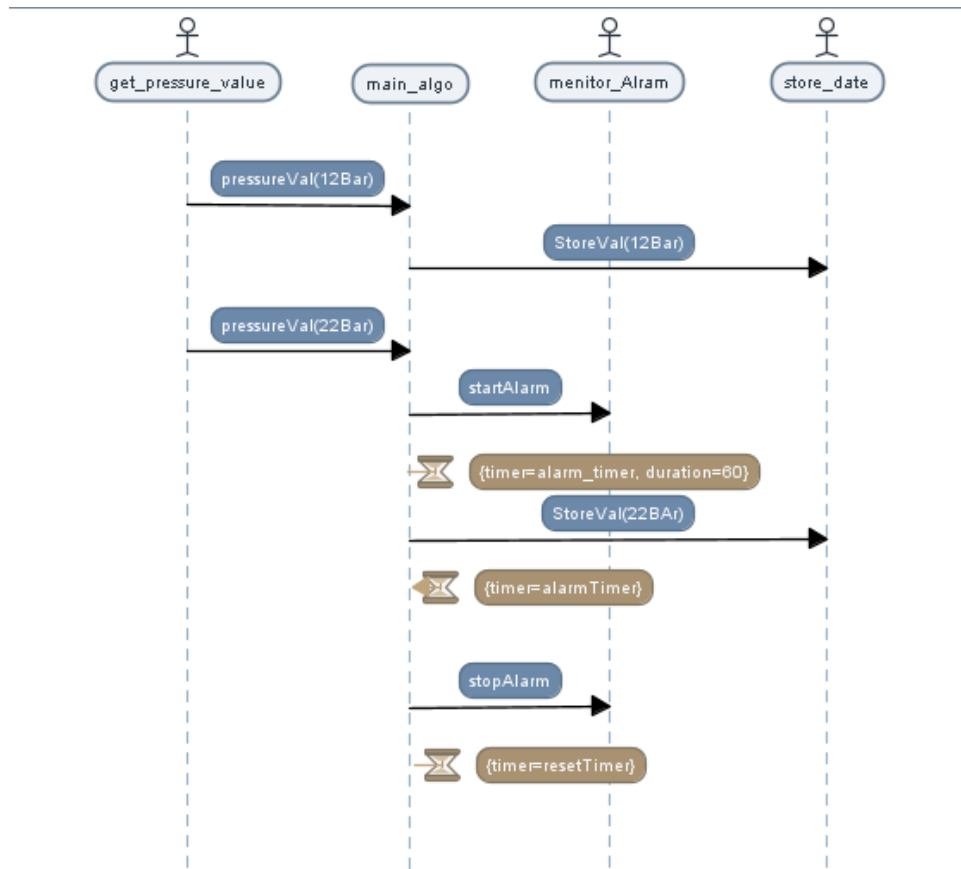


Figure 4. Sequence diagram for normal and high-pressure readings.

5. AVATAR System Design

5.1 Block Architecture

The AVATAR block design separates the system into four principal responsibilities. Pressure_Sensor_Driver acquires and publishes pressure measurements. main_algo stores the pressure threshold and decides whether an alarm request is required. Alarm_Monitor manages the alarm activation period and generates start/stop commands. Alarm_Actuator_Driver represents the hardware-facing actuator used to switch the alarm output.

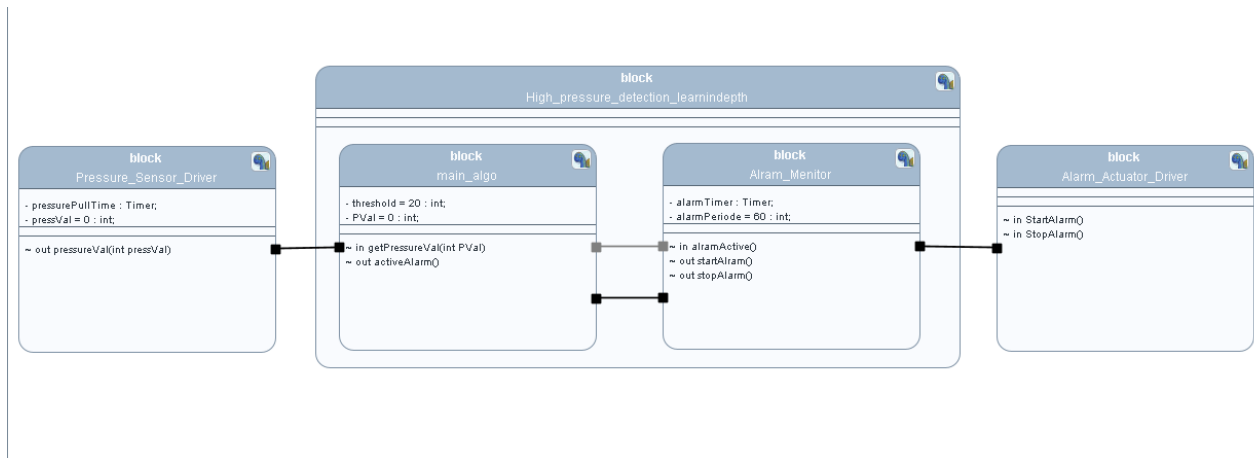


Figure 5. AVATAR block diagram and signal connections.

5.2 Main Algorithm State Machine

The main algorithm waits for a pressure value through `getPressureVal(PVal)`. The received value is compared with `threshold = 20`. Values below the threshold return to the sensing state without activating the alarm. When the modeled high-pressure condition is satisfied, the state machine sends `activeAlarm()` to the alarm monitor and then returns to the sensing loop.

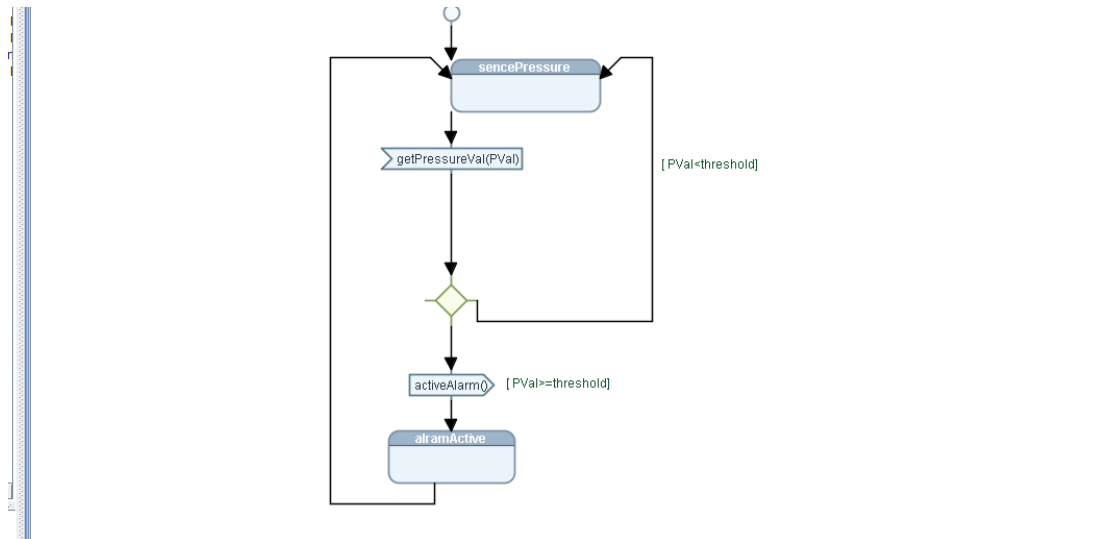


Figure 6. State machine of the main pressure-evaluation algorithm.

5.3 Alarm Monitor State Machine

The alarm monitor begins in an `AlarmOFF` condition. After receiving `alarmActive()`, it transitions to `AlarmON`, sends `startAlarm()`, and configures `alarmTimer` for 60 seconds. The monitor waits for the timer to expire, sends `stopAlarm()`, resets the timer, and returns to `AlarmOFF`. This structure isolates alarm timing from the pressure-evaluation logic.

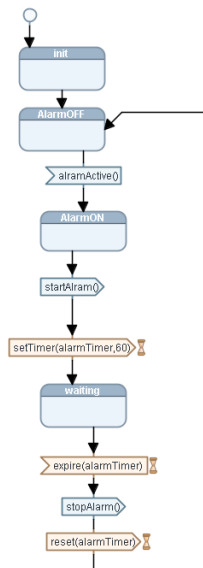


Figure 7. Complete alarm-monitor state machine.

5.4 Pressure Sensor Driver State Machine

The pressure-sensor driver models periodic acquisition. In the current TTool model, a simulated pressure value is generated in the range 15 to 25 and transmitted using `pressureVal(pressVal)`. A timer named `pressurePullTime` is then set to 100 seconds. After the timer expires, it is reset and the driver returns to `readFromSensor` for the next measurement.

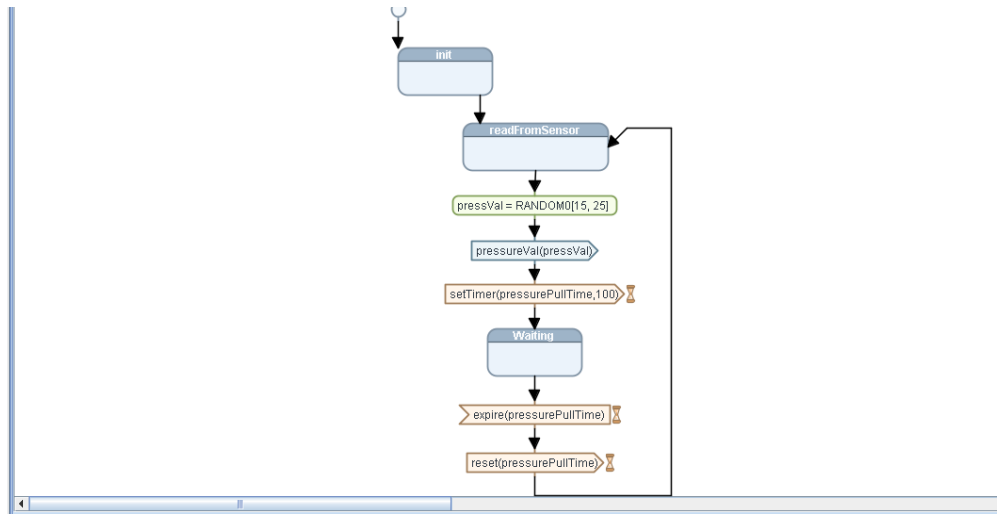


Figure 8. Pressure-sensor driver state machine with 100-second polling timer.

5.5 Alarm Actuator Driver State Machine

The alarm actuator driver waits for commands from the alarm monitor. A StartAlarm() event causes a transition through the Start state and then back to waiting, while StopAlarm() causes a transition through the Stop state and returns to waiting. In a hardware implementation, these transitions can be mapped to an alarm output such as an LED, buzzer, relay, or other warning device.

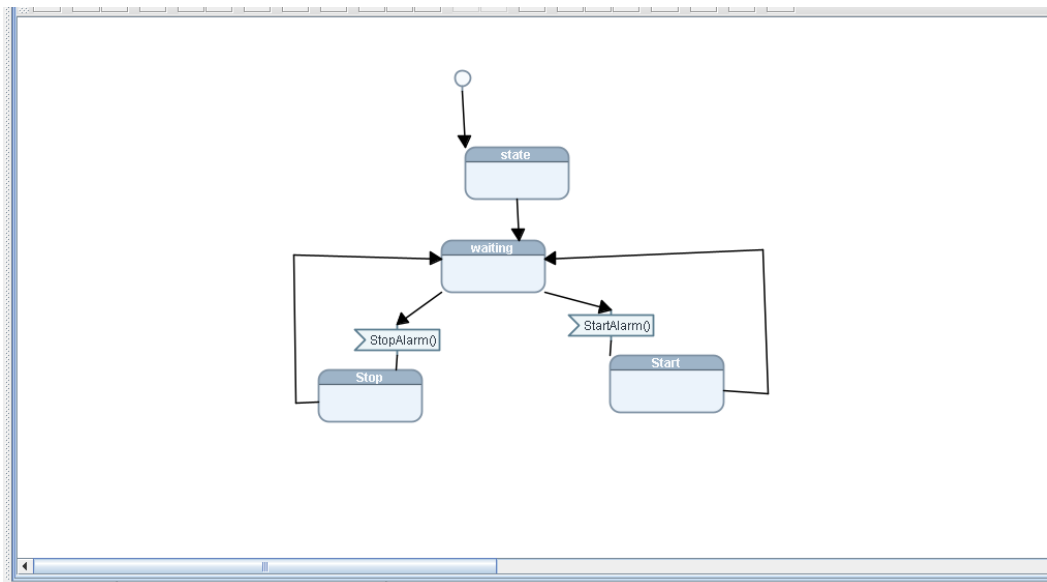


Figure 9. Alarm-actuator driver state machine.

6. Timing and Key Parameters

Parameter	Modeled value	Role in the system
Pressure threshold	20 bar	Configured in main_algo as threshold = 20.
Alarm duration	60 s	Configured by setTimer(alarmTimer, 60).
Sensor polling period	100 s	Configured by setTimer(pressurePullTime, 100).
Simulated sensor range	15-25 bar	Used in the pressure-sensor driver model.

Local storage	Optional	Represented in requirement, activity, use-case, and sequence views.
---------------	----------	---

7. End-to-End Operational Scenario

1. The pressure-sensor driver acquires a pressure reading and sends it to main_algo.
2. main_algo compares the reading with the configured pressure threshold.
3. If the reading is below the threshold, the system returns to monitoring without starting the alarm.
4. If the high-pressure condition is satisfied, main_algo sends activeAlarm() to Alarm_Monitor.
5. Alarm_Monitor sends startAlarm() to the actuator and starts a 60-second timer.
6. When the timer expires, Alarm_Monitor sends stopAlarm(), resets the timer, and returns to AlarmOFF.
7. The sensor driver independently waits for its 100-second polling timer before obtaining the next reading.
8. When enabled, pressure measurements can also be stored in local memory.

8. Implementation

The C implementation follows the same modular idea used in the TTool design. Instead of reproducing the full source code here, this section focuses on the module interfaces, inter-module APIs, and state-machine implementation that connect the model to the software.

8.1 Header APIs and Module Responsibilities

Module / Header	Main APIs / Interface	Purpose
Pressure Reader (read.h)	init(), readVal	Initializes and executes the pressure-reading state machine.
Main Algorithm (main_algo.h)	Main_Algo_init(), setPressureValToMain(int), main_algo	Receives the pressure value, compares it with the threshold, and selects the next state.
Alarm Monitor (alarmMintor.h)	MTR_init(), alarmAction(), alarmMtr	Receives the alarm request and controls the alarm state machine.
Low-Level Driver (driver.h)	GPIO_INITIALIZATION(), getPressureVal(), Set_Alarm_actuator(int), Delay(int)	Provides the GPIO and hardware-facing functions used by the software modules.
State Framework (state.h)	STATE_define(), STATE()	Macros used to declare and reference state functions.

8.2 Inter-Module APIs and TTool Signal Mapping

The most important interfaces are the APIs that transfer events or data between modules. These correspond directly to the signals shown in the TTool block diagram.

Software connection	C API	TTool equivalent	Action
Pressure Reader -> Main Algorithm	setPressureValToMain(int p)	pressureVal / getPressureVal	Passes the measured pressure to the decision logic.
Main Algorithm -> Alarm Monitor	alarmAction()	activeAlarm()	Requests alarm activation when pressure is above the threshold.
Alarm Monitor -> Alarm Actuator	Set_Alarm_actuator(1 / 0)	startAlarm() / stopAlarm()	Sets or clears the alarm output (LED in the prototype).

8.3 State-Machine Implementation

Module	Implemented states	Implementation mechanism
Pressure Reader	readWaiting, sensorReading	readVal is a function pointer to the active state.
Main Algorithm	sencePressure, alaramActivation	main_algo points to the active decision state.
Alarm Monitor	alarmOFF, alarmON, waiting	alarmMtr points to the active alarm state.
Alarm Actuator	Start / Stop action	Implemented directly by Set_Alarm_actuator() rather than a separate C state-machine file.

State representation: No struct-based state object is used in this version. The state machines are represented by function pointers (readVal, main_algo, alarmMtr), enum state names, and module variables such as pressureVal, threshlod, and alarmPeriode.

Main execution order: after GPIO and state initialization, the main loop repeatedly executes readVal() -> main_algo() -> alarmMtr().

Model correspondence note: The structural data/event flow matches the TTool architecture. At exactly 20 bar, the current C code keeps the system in the normal path and activates the alarm only for values greater than 20; therefore, the TTool guard should use PVal > threshold for exact consistency.

9. Testing and Validation

Validation was performed at build/link level, memory-map level, symbol level, and by step-through debugging in Keil. The purpose was to verify that the custom startup code, linker script, build flow, and state-machine control sequence are connected correctly.

9.1 Build, Linker, and Memory-Map Validation

Check	Observed evidence	Result
Target memory	FLASH: 0x08000000, length 0x8000 (32 KB); SRAM: 0x20000000, length 0x2800 (10 KB).	Matches linker_script.ld.
.text / vectors	.vectors starts at 0x08000000; .text starts at 0x08000000 and ends at _E_text = 0x08000384.	Correctly located in FLASH.
.data	_S_DATA = 0x20000000 and _E_DATA = 0x2000000C; initialized values are linked for execution in SRAM with a Flash load image.	Consistent with SRAM AT> FLASH.
.bss / stack	_S_BSS = 0x2000000C, _E_BSS = 0x20000024, and _stack_ptr = 0x20001024.	BSS and reserved stack are placed in SRAM.

Startup validation: Reset_Handler uses the linker symbols to copy the initialized .data image from Flash to the SRAM .data region, clears the .bss range to zero, and then calls main(). This matches the memory layout produced by the linker.

9.2 Symbol Validation

The ELF symbol table was checked with arm-none-eabi-nm. The linker-defined symbols (_E_text, _S_DATA, _E_DATA, _S_BSS, _E_BSS, and _stack_ptr) are present together with Reset_Handler, the module APIs, and the generated state functions. This confirms that the required symbols were resolved into the final ELF image.

9.3 Keil Debug and State-Sequence Validation

Step-through debugging in Keil was used to follow the software sequence. After initialization, execution returns to the main loop and calls readVal(), main_algo(), and alarmMtr() in order. During the recorded test, a pressure value of 22 reaches setPressureValToMain(), selects ST_alamActivation, and transfers the alarm request to the Alarm Monitor. This verifies the intended high-pressure control path at state-machine level.

